

Introduction to Scalability

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

English:

Scalability is a system's ability to handle growing load -- more users, more requests, more data -- without a proportional degradation in performance. There are exactly two ways to add capacity to a system.

Hindi:

Scalability एक system की वह क्षमता है जिससे वह बढ़ते हुए load -- ज़्यादा users, ज़्यादा requests, ज़्यादा data -- को बना performance में उतनी ही गिरावट के संभाल सके। किसी system में capacity जोड़ने के ठीक दो तरीके होते हैं।

Important Words:

- **Degradation** – गिरावट / बगिड़ना
- **Proportional** – अनुपातिक

Vertical scaling (scale up)

English:

It helps to think of these two options as sitting on a spectrum of operational complexity. Every system starts simple, usually on a single machine, and the honest first question is never 'how do I scale to a billion users' -- it's 'at what point does this design actually need to change.'

Hindi:

इन दोनों विकल्पों को operational complexity के एक spectrum पर सोचना मददगार होता है। हर system शुरुआत में simple होता है, आमतौर पर एक ही machine पर, और सबसे ईमानदार पहला सवाल कभी यह नहीं होता 'मैं इसे अरबों users तक कैसे scale करूं' -- बल्क यह होता है 'यह design असल में किस point पर बदलने की ज़रूरत पड़ेगी।'

English:

Make the existing machine bigger: more CPU, more RAM, faster disks. It's simple but has a hard ceiling, it's a single point of failure, and it usually costs disproportionately more as you approach that ceiling.

Hindi:

मौजूदा machine को बड़ा करे: ज़्यादा CPU, ज़्यादा RAM, तेज़ disks। यह सरल है लेकिन इसकी एक सख्त सीमा (ceiling) होती है, यह एक single point of failure है, और उस सीमा के पास पहुंचते-पहुंचते इसकी लागत बहुत ज़्यादा बढ़ जाती है।

Important Words:

- **Ceiling (figurative)** – अधिकतम सीमा

Horizontal scaling (scale out)

English:

Add more machines and distribute load across them. This has effectively no ceiling, and failure of one machine doesn't take down the whole system. The cost is complexity: your application must work correctly when spread across many machines.

Hindi:

ज्यादा machines जोड़े और load को उनमें बांट दे। इसकी व्यावहारिक रूप से कोई सीमा नहीं होती, और एक machine के fail होने से पूरा system नहीं गिरता। इसकी कीमत complexity है: आपके application को सही तरीके से काम करना होगा जब वह कई machines में फैला हो।

Backend engineer analogy

English:

Upgrading your production server from 4GB to 32GB RAM is vertical scaling. Running your Laravel app on 10 separate servers behind a load balancer is horizontal scaling. Most engineers reach for vertical scaling instinctively first; HLD interviews almost always expect horizontal scaling as the real answer.

Hindi:

अपने production server को 4GB से 32GB RAM तक upgrade करना vertical scaling है। अपने Laravel app को एक load balancer के पीछे 10 अलग-अलग servers पर चलाना horizontal scaling है। ज्यादातर engineers पहले सहज रूप से vertical scaling की तरफ जाते हैं; HLD interviews लगभग हमेशा horizontal scaling को ही असली जवाब मानते हैं।

The precondition for horizontal scaling: statelessness

English:

Horizontal scaling only works cleanly if any server can handle any request. A stateless service keeps all shared state externally -- in a database, a cache, or a token -- so it doesn't matter which server instance handles which request.

Hindi:

Horizontal scaling तभी सही तरीके से काम करता है जब कोई भी server किसी भी request को संभाल सके। एक stateless service अपना सारा shared state बाहर रखती है -- किसी database, cache, या token में -- ताकियह मानने ना रखे कि कौन सा server instance कौन सी request handle कर रहा है।

Important Words:

- **Precondition** – पूर्व-शर्त
- **External (state)** – बाहरी

English:

In practice, horizontal scaling is rarely done manually -- cloud platforms support auto-scaling: rules that automatically add or remove server instances based on live metrics like CPU utilization.

Hindi:

व्यवहार में, horizontal scaling शायद ही कभी manually की जाती है -- cloud platforms auto-scaling support करते हैं: ऐसे rules जो CPU utilization जैसी live metrics के आधार पर अपने आप server instances जोड़ते या हटाते हैं।

Important Words:

- **Metric** – मापदंड / मीट्रिक

2. Real-World Example / Case Study

English:

During Careem's peak commute hours, their application tier scales horizontally -- auto-scaling adds more stateless application server instances as traffic rises, then removes them as traffic falls, keeping cost proportional to demand.

Hindi:

Careem के peak commute hours के दौरान, उनका application tier horizontally scale होता है -- auto-scaling traffic बढ़ने पर ज़्यादा stateless application server instances जोड़ती है, फिर traffic घटने पर उन्हें हटा देती है, ताकि cost demand के अनुपात में रहे।

English:

Meesho's catalog database, by contrast, cannot simply 'add more servers' the same casual way -- scaling a relational database horizontally requires deliberate techniques like read replicas and sharding.

Hindi:

इसके विपरीत, Meesho का catalog database इतनी आसानी से 'और servers जोड़ना' नहीं कर सकता -- एक relational database को horizontally scale करने के लिए read replicas और sharding जैसी सोची-समझी techniques चाहिए होती हैं।

Important Words:

- **Casual (adverb, figurative)** – बिना ज़्यादा सोचे-समझे

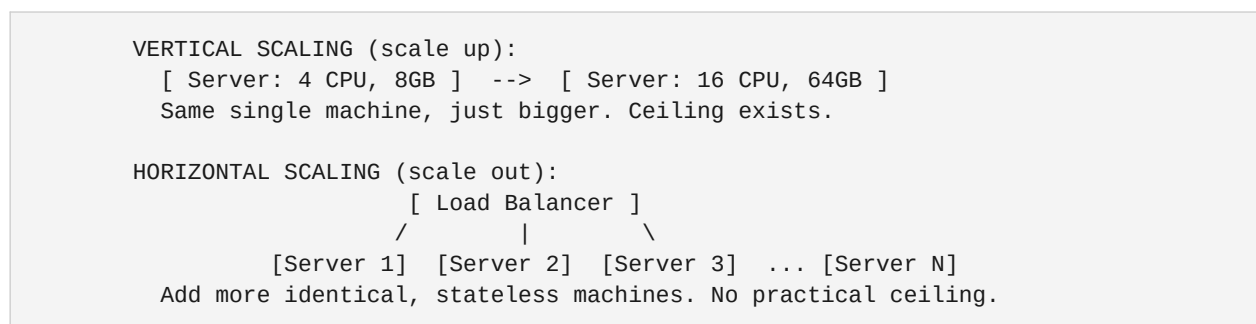
English:

A second example: Zomato/Swiggy's order-placement service scales horizontally across lunch and dinner peaks, while their catalog database has historically leaned more on vertical scaling and read replicas.

Hindi:

एक और उदाहरण: Zomato/Swiggy की order-placement service lunch और dinner के peaks में horizontally scale होती है, जबकि उनका catalog database पहले से ज़्यादातर vertical scaling और read replicas पर निर्भर रहा है।

3. Diagram (English labels kept as-is)



4. Trade-offs Table (Reference Table – English)

Aspect	Vertical Scaling	Horizontal Scaling
Complexity	Low -- no code changes needed	Higher -- requires stateless design

Aspect	Vertical Scaling	Horizontal Scaling
Ceiling	Hard limit	Effectively unlimited
Failure impact	Single point of failure	One instance failing doesn't break the system
Typical first move	Quick fix for a spike	The durable, standard HLD answer

5. Common Interview Questions

Q1: "How would you scale this service to handle 10x traffic?"

English:

Model approach: Default to horizontal scaling of the app tier, confirm the service is stateless first, then separately address whether the database can keep up.

Hindi:

Model approach: app tier की horizontal scaling को default माने, पहले पक्का करें कि service stateless है, फिर अलग से यह देखें कि database इसे संभाल पाएगा या नहीं।

Q2: "Why can't we just keep buying a bigger database server forever?"

English:

Model approach: Explain the hard ceiling and steep cost curve of vertical scaling, and that it remains a single point of failure no matter how big it gets.

Hindi:

Model approach: vertical scaling की सख्त सीमा और तेज़ी से बढ़ती लागत को समझाएं, और यह भी कहें कि कितना भी बड़ा हो जाए फिर भी single point of failure ही रहता है।

Q3: "How would auto-scaling decide when to add or remove servers?"

English:

Model approach: Describe metric-based rules with thresholds and a cooldown period. Scaling out is made aggressive/fast; scaling in is made more conservative/slow.

Hindi:

Model approach: threshold और cooldown period वाले metric-based rules बताएं। Scaling out को तेज़/aggressive रखा जाता है; scaling in को धीमा/conservative रखा जाता है।

Important Words:

- **Cooldown period** – कूलडाउन अवधि

6. Practice Exercises

English:

- Explain why a service storing sessions in local server memory cannot be horizontally scaled without changes.
- For a system you've worked on, identify one component scaled vertically and one needing horizontal scaling.
- Draw the horizontal scaling diagram from memory, including the load balancer.

Hindi:

- समझाएं कि Sessions को local server memory में रखने वाली service बिना बदलाव के horizontally scale क्यों नहीं हो सकती।
- अपने किसी project में एक component बताएं जो vertically scale हुआ और एक जैसी horizontal scaling की ज़रूरत पड़ेगी।
- load balancer सहित horizontal scaling का diagram याद से बनाएं।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"System Design Fundamentals: Vertical Scaling vs Horizontal Scaling"	Interview-oriented explanation.
English	"Horizontal and Vertical Scaling (How to Scale Your Application)"	Practical framing.
Hindi	"Horizontal Vs Vertical Scaling System Design Simplified Hindi"	Uses real-world analogies.
Hindi	"Horizontal or Vertical Scaling? Explained in Hindi" (Day 6/100)	Concise Hindi comparison.

8. Key Takeaways

English:

- Vertical scaling = bigger machine; horizontal scaling = more machines.
- Vertical scaling is simple but has a hard ceiling and remains a single point of failure.
- Horizontal scaling has no practical ceiling but requires the service to be stateless.
- Stateless services externalize shared state so any instance can handle any request.
- Application-tier scaling and database scaling are different problems.
- HLD interviews almost always expect horizontal scaling as the default, durable answer.

Hindi:

- Vertical scaling = बड़ी machine; horizontal scaling = ज़्यादा machines।
- Vertical scaling सरल है लेकिन इसकी सख्त सीमा होती है और यह single point of failure बना रहता है।
- Horizontal scaling की व्यावहारिक रूप से कोई सीमा नहीं, लेकिन इसके लिए service का stateless होना ज़रूरी है।
- Stateless services अपना shared state बाहर रखती हैं ताकि कोई भी instance कोई भी request संभाल सके।
- Application-tier scaling और database scaling अलग-अलग समस्याएं हैं।
- HLD interviews लगभग हमेशा horizontal scaling को ही default, टिकाऊ जवाब मानते हैं।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Casual (adverb, figurative)	बना ज़्यादा सोचे-समझे	Done without much special effort
Ceiling (figurative)	अधिकतम सीमा	The upper limit that can't be crossed
Cooldown period	कूलडाउन अवधि	A waiting time before taking another scaling action
Degradation	गरिबट / बगिड़ना	A decline in quality or performance
External (state)	बाहरी	Stored outside the individual server
Metric	मापदंड / मीट्रिक	A measurable value used to track performance
Precondition	पूर्व-शर्त	A condition that must be true before something else can work
Proportional	आनुपातिक	Changing at the same rate as something else